PRESENCE

The business side of personal agents

Your next customer is an agent.
Your business should be ready.

Research-backed plan for Adobe's
The Post-Website Business challenge

31 October 2026 | Europe/Zurich | Reviewed planning version 2

Three-minute pitch. Three demo moments.
Publish an approved presence. Complete a request through email. Reconnect with permission.

Scope: one fictional Basel hotel; one shared Next.js application; versioned tenant configurations; Docker Compose.

Status: planning documents only. No application, benchmark results or measured revenue uplift. Pricing and financial examples are hypothetical.

31 October 2026. Review of the proposed product, demo, user flows and architecture; no application has been implemented or tested.

Verdict

The commercial idea and email-first adoption path make sense. Version 1 was a strategy brief, not yet a build contract. Its weakest points were an undefined presentation experience, mixed demo/future boundaries, an unresolved durability choice, and insufficient detail around who can confirm a booking. These are corrected in version 2 and the [implementation specification](implementation-spec.md).

Confirmed direction

The user clarified that the deliverable is for the final presentation and that **URL onboarding, booking through real email, and an opted-in agent follow-up must all be live**. The selected interface is a standalone Next.js application using Tailwind and shadcn/ui. It is not an embedded ChatGPT application. The presenter operates the demo; owner and traveller remain separate domain actors with separate authority.

The user chose to proceed without supplying a deadline. No submission time is assumed. Build the three connected moments first and reduce experiment breadth and roadmap features before cutting any of those moments. Real mail and model credentials are setup dependencies, not an invitation to invent successful results.

Critical issues and corrections

P1. The user interface had no concrete product flow

Version 1 listed an owner dashboard and an optional traveller chat without explaining how the audience would follow them. A builder could have produced disconnected screens, generic analytics cards or a chat-only prototype.

Correction: a single presentation route with three fixed chapters, a clear owner/traveller context switch, a task-focused conversation and a persistent business/offer panel. Structured cards handle publication, approval, terms and subscription; chat supports the work. Screen states, component choices, visual tokens and responsive behavior are specified. See specification sections 1-4.

P1. External buyer agents and our demo buyer were conflated

The old context diagram described an external buyer platform, while the container notes put its loop in our worker. That obscured what interoperability the prototype could actually prove.

Correction: the running prototype contains its own buyer-agent simulator with an isolated principal. It invokes the public merchant API, not private service shortcuts. Future external buyer platforms are dashed integrations. The context, container and component diagrams now use that same boundary. The component view decomposes the web/API container only; worker responsibilities are shown as an adjacent container.

P1. Durable execution was not selected

The old plan relied on an unspecified self-hosted Workflow World and also proposed a fallback queue. The required worker and database connections could have been discovered too late.

Correction: select PostgreSQL plus pg-boss for jobs, with explicit persisted domain state and an outbox. Human waits occupy database records, not a live LLM loop. A durable queue is not an exactly-once guarantee for email or bookings. The commercial transaction and outbox share a SQL transaction; a dispatcher schedules jobs from committed outbox records, so queue/database commits

do not need to be magically atomic. Workflow SDK remains a future alternative, not a second runtime to build now. [S28]

P1. Offer acceptance and booking confirmation were underspecified

A request status was used to stand for offers, holds and reservations. A real email-only hotel cannot atomically reserve inventory just because a customer accepts an offer.

Correction: separate inquiry, immutable offer revision, reservation attempt, confirmation evidence and delivery state. In the fictional inventory adapter, acceptance validates and commits capacity atomically. In an email-only adapter, acceptance creates a pending reservation and a second owner-confirmation step; confirmation requires an owner action or authoritative external reference. UI status and agent language follow the selected adapter.

P1. Real email was simultaneously mandatory and optional

Version 1 offered a dashboard shortcut and suggested demonstrating real inbound mail only in Q&A. That fails the clarified scope.

Correction: the on-stage booking flow includes an actual outbound email and an actual owner reply through Resend inbound. Parsed mail becomes a proposed offer; an authenticated owner action approves it. An email client is prepared before presenting. The dashboard is for explicit approval and recovery, not a substitute for claiming the email round trip occurred. Provider message IDs and inbound receipt are inspectable; mail delivery time is a real external dependency.

P1. Authority was missing from the presentation controls

One browser operating both sides can accidentally become a single omnipotent account. A role label in the UI provides no authorization.

Correction: Better Auth manages the presenter session; explicit owner membership and a separate seeded buyer principal govern actions. Demo orchestration is enabled only for seeded demo tenants behind a server-side feature flag and operator permission. Production owner actions and buyer actions retain independent authorization and audit identities. Role switching changes the visible context, not the underlying permissions. [S29]

P1. The benchmark and protocol scope could consume the hackathon

432 episodes, multiple protocol adapters, a full growth dashboard and three live moments are too much to promise before the basic slice works. Base-model comparisons also cannot demonstrate immediate organic search lift.

Correction: REST/OpenAPI and a real MCP tool adapter share one commercial service. A2A and UCP are deferred adapters. Run six core scenarios, two model families, three repeats and two arms for 72 exploratory episodes after smoke checks. The 12-task/four-arm matrix remains an expansion. Keep three live demo moments mandatory and show only measured results.

P2. Technical structure was too vague to constrain code quality

The old plan did not select an authentication library, specify transaction/queue boundaries, distinguish chat streaming from task events, or define acceptance tests and module ownership.

Correction: the implementation specification fixes those choices, error semantics, actor access, schemas, job handlers, replay behavior and a feature-oriented directory layout. Visual and functional exit checks are explicit. The product cannot be called complete on the basis of a successful compilation.

P2. Two separately maintained diagram representations could drift

The Markdown Mermaid and the PDF renderer used separate edge definitions. The rendered level 3 also combined consent and evaluation even though they are different responsibilities.

Correction: architecture node/edge data and layout are stored in `architecture.json`; the same data generates the Mermaid blocks and rendered PDF. Consent and evaluation are separated. Diagram relationships include direction and technology; a single arrow is not described as bidirectional unless both directions are drawn.

P2. The pitch had little allowance for live actions

The original 359-word speaker draft left insufficient room for extraction, real mail and approvals inside three minutes.

Correction: reduce spoken copy, preload only non-demonstrated setup, and use one continuous UI. Targets are rehearsal goals, not guaranteed provider latency. Preflight real mail, keep an honest failure/retry state, and retain a labelled recording for external outages; a replay does not count as the required live proof.

Review exit

The documentation is implementation-ready under the clarified standalone-demo scope. Application readiness is not established. Before coding the full UI, verify the real email round trip, harness access and durable job restart behavior; then implement and rehearse the connected flow. No more product-choice questions are pending from this review.

01. The decision we should align on

Jury takeaway: When customers send an AI, independent businesses need a way to participate. We turn their existing website into a business-controlled commerce interface, connect it to the inbox they already use, and preserve a permission-based path to repeat business.

Proposed pitch line: Your next customer is an agent. Your business should be ready.

The product is a shared, multi-tenant commercial layer with three visible outcomes: **publish, transact, reconnect**. Its commercial buyer is the independent hotel owner; its hackathon interface is designed for the presenter and audience. Owner and traveller remain separate authorized domain actors. The prototype includes our own clearly identified buyer-agent simulator, which uses the public merchant API. Independent external assistants are a future integration, not an assumed completed capability. Start with one fictional Basel hotel and one traveller; restaurant support is a later template.

The strongest demonstration is a nonstandard request that a generic listing cannot reliably resolve: a traveller arrives after midnight with a dog and needs a quiet workspace after checkout, under CHF 220. The hotel cannot promise late checkout but can offer a room plus workspace. The owner approves the offer through email or a structured dashboard. The customer approves the exact terms. A real database records the resulting reservation.

Publishing an API alone does not make a hotel appear in every assistant. The demo must say how the agent finds the business. Use an explicit local directory for the controlled test; treat organic discovery as a separate, slower experiment. The prototype is a working commercial workflow; a chat can expose the traveller's actions but is not the product's entire value.

The actual challenge

Recovered from the other chat, titled **Explore Adobe's post-website future**, and its attached challenge text. The official title is **The Post-Website Business**, sponsored by Adobe, posted 10 October 2026. The spoken labels "hope" and "5 challenge" do not match that title; the recovered attachment is the planning basis. [S1]

The brief asks how local businesses establish and control their presence when the visitor is an AI agent. It requires a believable future in roughly 5-10 years and a running prototype of a real slice. Agent-facing offerings, discovery, trust, transactions, owner control, fairness, and human fallback are suggested directions, not a mandatory checklist. The brief explicitly prefers an honestly scoped working slice. It requests an OSI-standard license in the submission repository. [S1]

2030 is our near-term scenario, four years from this event. Explicitly extend the vision to **2031-2036** on the roadmap to cover the brief's 5-10-year framing. The future is a scenario, not a forecast with a guaranteed adoption rate.

What success looks like

- The jury can explain the customer, the problem, and the change after one sentence.
- Three connected demo moments work: onboard from a URL; complete an owner-approved request; deliver an opted-in event to the same traveller agent.
- Every promised price, booking status, and benchmark result has a traceable source or persisted record.
- The before/after evidence is labelled as a controlled experiment. Financial illustrations are visibly hypothetical.

02. Preserve the full concept, then choose the slice

The user's vision, recorded

By 2030, personal agents may combine preferences, identity, delegated spending authority and ongoing context. They will compare more providers, solicit offers, and act within boundaries chosen by a person. That may shift influence over the customer relationship toward the assistant platform. Independent businesses need a voice in those comparisons and a way to earn repeat business.

Our adoption path begins with the information a business already has: a domain, business name, Google Maps link, or contact number. The system extracts facts, identifies missing operational information, interviews the owner only where needed, and publishes an agent-readable presence plus a transactional interface. Agent requests become normal emails to the business; replies become structured offers and task updates. A small dashboard gives the owner an alternative to writing email.

Over time, CRM, PMS, booking engine, POS and ERP adapters replace manual steps. These systems have different roles: PMS/booking engines govern hotel inventory; POS/reservation systems govern restaurant capacity; CRM handles relationships; ERP contributes finance and fulfilled-revenue data. Do not promise that an ERP connection alone provides bookable availability.

The long-term service continuously tests business descriptions, capability schemas, response policies and offers across several buyer agents and models. It connects representation and selection to inquiries, accepted offers, completed bookings, cancellations, repeat visits and contribution margin. Models can propose experiments; the business controls publication and commercial authority.

With customer consent, a hotel or restaurant can publish relevant updates to subscribed personal agents: an event, a suitable room becoming available, or a return-visit offer. The personal agent decides whether the update matters and whether to show it. The merchant gains a permission-based connection rather than a right to insert arbitrary instructions into someone else's agent.

Adoption stages

- **A. Publish:** verified identity, sourced facts, distinctive brand material, policies, clear unknowns, machine-readable capabilities, and a human page from the same data.
- **B. Email bridge:** asynchronous inquiries and offers; owner remains the authority; no unsupported real-time availability claims.
- **C. Connected operations:** PMS/POS and CRM adapters; live quotes and resource holds where the underlying system supports them.
- **D. Measured growth:** experiments linked to fulfilled outcomes; owner-approved routing and offers, with capacity and margin guardrails.
- **E. Permission-based retention:** relevant events to subscribed customer agents; revocable consent, visible preferences and delivery receipts.

Deliberate assumptions

The agent acts on behalf of a human or company under delegated authority; it is not assumed to be a new legal person. Payment and identity are handled by specialist providers later. Broad consumer adoption, universal webhook support, and the assistant platform's dominance of the relationship remain hypotheses. The names "Muse" and "Dropbot" in the spoken request are ambiguous; the pitch should use verified products and protocols rather than attribute unverified features to those names.

03. Evidence, trends, and market size

Evidence we can actually cite

Adobe reported that AI-referred traffic to US travel sites grew **194% year over year in May 2026**. Its analysis covered more than 8 million visits to US travel sites. Crucially, this traffic still converted **28% less** than non-AI traffic. The direction is promising; this does not show that agents already dominate bookings or that the same effect applies to Swiss hotels. [S2]

UCP announced a **draft lodging-booking capability on 25 September 2026**, with participation from established travel and hospitality companies. That is evidence of infrastructure investment, not universal implementation or a final standard. [S3]

Stripe and OpenAI's ACP work demonstrates programmatic commerce between buyer agents and merchants. Its retail checkout context is not evidence that every hotel can expose inventory or accept autonomous reservations today. [S4]

A2A supports asynchronous task updates, including authenticated client webhooks when the server advertises push notifications. This makes an email-to-agent bridge technically plausible today. Availability in a protocol does not mean every personal assistant implements it. [S5]

Bottom-up market framing

Swiss hotel beachhead: the FSO's Swiss Tourism Statistics 2024 states that **3,999 hotels and health establishments were open in 2024**. Use approximately 4,000 as an order-of-magnitude establishment base, not a count of independent, qualified software buyers. The report's adjacent chart gives 4,005, so use the prose figure with its year and definition rather than claim false precision. Establishments and enterprises are different units. [S6]

European expansion ceiling: Eurostat reports **1.988 million enterprises in accommodation and food services in 2023**, in an article extracted in May 2026. This includes a broad mix of businesses and is not a count of independently owned hotels with suitable demand. [S7]

Pricing hypothesis: CHF 149 per business per month for a pilot offer, with assisted onboarding initially. At that assumed price, 3,999 Swiss establishments imply an upper-bound annual subscription envelope of **CHF 7.15 million**, before qualification, adoption, discounts or churn. A hypothetical 25% eligible segment is about 1,000 businesses and CHF 1.79 million annual revenue capacity. A reachable first-year target of 50 paying businesses would be **CHF 89,400 ARR**. These are arithmetic scenarios, not validated SAM, pipeline or forecasts.

At an independently assumed EUR 149 per enterprise per month, the broad EU count implies a **EUR 3.55 billion annual envelope**. Currency and buyer-unit differences prevent adding it to the Swiss estimate. This ceiling belongs in backup material; the three-minute pitch should lead with the beachhead and a pilot hypothesis.

What remains unproven

Interview 3-5 independent hotels and one hospitality agency: how many inquiries arrive by email, what details require human judgment, where booking authority lives, what a missed inquiry costs, and whether the owner would pay for this. Ask the Adobe mentor which presence-authoring and distribution workflows matter most. Small traffic today may support operational efficiency as the first buyer benefit before agent-attributed growth becomes material.

04. Competitive landscape: five useful comparisons

These are overlaps based on public descriptions, not exhaustive feature audits. Do not claim that any competitor lacks a feature merely because a page does not mention it.

- **Adobe Brand Visibility, formerly LLM Optimizer:** enterprise AI visibility, representation, optimization, experimentation and attribution. It overlaps substantially with the planned optimization loop. Our proposed extension is a small-business path from approved facts to operational transactions through email. Present this as a possible Adobe workflow, without suggesting a partnership. [S8]
- **Profound:** its Answer Engine Insights queries answer engines and analyzes how brands appear. The lesson is repeatable prompts and response evidence. Our prototype also executes merchant-approved workflows; mention counts alone are not its success criterion. [S9]
- **Wildcard:** YC describes a platform for agentic commerce optimization for ecommerce and retail, including ACP/UCP support. This is a direct adjacent competitor; "we make brands agent-ready" is insufficient differentiation. Test the hospitality-specific email bridge and permission-based repeat relationship as the wedge. [S10]
- **Palisade:** YC describes AI-native sales agents that run marketplaces. The other chat also identified overlap in persistent sales conversations; that finer feature comparison has not been re-verified from readable primary text here. Treat it as a competitor to investigate, not evidence of an unoccupied category. [S11]
- **Stripe / ACP:** transaction and payment infrastructure that can be a partner or dependency. We should adopt standards where relevant rather than invent payment rails. Our first slice resolves manual merchant operations before payment. [S4]

Positioning to defend

An adoption and operating layer for independent hospitality: publish once, answer agents through your existing inbox, and reconnect through customer-approved subscriptions. The long-term advantage would be useful operational adapters, verified merchant data, and outcome-linked experiments. Those are hypotheses about defensibility; none is a moat at hackathon scale.

UCP, ACP, MCP and A2A are different building blocks. MCP exposes tools; A2A coordinates tasks between representatives; commerce protocols define commercial flows. A manifest or OpenAPI document helps an integrated agent use our service but does not automatically register it with every assistant. Keep a shared internal contract and add adapters rather than building four separate backends.

05. Three-minute pitch: one bullet per slide

- **1 | 0:00-0:18 | From answers to action:** one visual timeline from the 1956 AI research era to the 2017 Transformer, 2022 ChatGPT, 2026 commerce protocols, and our clearly labelled 2030 scenario; ask, "What does a business publish when its customer sends an agent?" [S3, S4, S21, S22]
- **2 | 0:18-0:35 | The owner loses the interface:** role-play a fictional Basel hotel owner whose rooms fit a traveller but whose policies and booking process cannot be reliably resolved by the agent; show the failed task and its actual reason.
- **3 | 0:35-1:05 | Demo: publish:** enter the hotel's URL, inspect the sourced business profile, confirm one missing policy, and publish a human page plus machine-readable capabilities from the same record.
- **4 | 1:05-1:45 | Demo: transact:** the traveller's agent requests late arrival, a dog, and workspace after checkout; the owner approves a CHF 205 room-plus-workspace offer by dashboard or email, the traveller approves, and a persisted reservation appears with exact terms.
- **5 | 1:45-2:10 | Demo: reconnect:** the traveller opts into suitable future updates, the owner publishes an event, and the agent receives it and offers a relevant suggestion; show the subscription and an unsubscribe control.
- **6 | 2:10-2:35 | Evidence and business:** show actual controlled before/after task results, the approximately 4,000-establishment Swiss beachhead, and the CHF 149/month pilot hypothesis; clearly distinguish measured outcomes from the financial illustration. [S6]
- **7 | 2:35-3:00 | Close the loop:** cite AI travel referral growth and the UCP lodging draft, show the email-to-PMS adoption path, and close, "The next web is agents acting for people. We help local businesses stay in the relationship." [S2, S3]

Total: **180 seconds**, including all demo actions. Use seven views in a continuous presentation; avoid repeated switches between unrelated windows. Keep AI history to 18 seconds. If a demo stalls for more than five seconds, switch to the prepared, clearly labelled replay.

Speaker draft - three-minute narrative

AI has moved from research to conversation, and now toward action. Imagine 2030: your family asks an assistant to arrange a weekend. It compares businesses and requests offers for you.

What happens to an independent hotel that still handles special requests by email? Its rooms might be perfect, but the agent cannot reliably resolve its policies or obtain an offer.

Presence is the business side of personal agents.

Here is our fictional Basel hotel. We enter its website, inspect the extracted facts and confirm a missing policy. One approved record now serves people and agents.

Our traveller arrives after midnight with a dog and needs workspace after checkout, under 220 francs. Watch the request reach the owner's actual inbox. Their reply becomes an offer: a room and workspace for 205. The owner approves the terms; the traveller accepts; here is the confirmed fictional reservation.

The relationship continues only with permission. The traveller subscribes to suitable events. The hotel publishes one; their agent receives it and suggests returning. Unsubscribe remains under the traveller's control.

Our controlled tests compare completed tasks before and after publication. They do not claim instant search ranking. Switzerland has roughly four thousand open hotel establishments; 149 francs per month is our pilot pricing hypothesis.

AI travel referrals are growing, and a lodging commerce standard is already in draft. Start with a website and an inbox; connect operational systems next. When customers send agents, local businesses should stay in the relationship.

06. Demo contract: three moments, one continuous story

A. Publish an existing business

Input: a fictional hotel's locally hosted source page, also accessible by URL. The system actually fetches the page and extracts facts at runtime. Before publication it shows source URLs, observation times, confidence, and unknowns. One short owner question resolves workspace access. A readable brand page, JSON business profile, OpenAPI description and MCP tools are projections of the same approved version. No new application or deployment is generated.

The old page stays available as a human fallback. The owner can update a fact once and preview both representations. Do not use fabricated review scores or add an unsupported "quiet" claim just to win the test. If claiming distance from Basel SBB, use fixture coordinates and a clearly specified distance method or owner-provided evidence.

B. Handle a request through the inbox

Traveller prompt: "I arrive after midnight with my dog. I need a quiet place to work until 14:00 after checkout. Keep the stay and extras below CHF 220. Ask me before confirming."

The buyer agent chooses tools. A genuine request record creates a merchant task. The owner receives the requirements and a link to the structured response form, plus an email reply option. Offer: CHF 175 room, CHF 20 dog fee, CHF 10 workspace, **CHF 205 total**, with fixture taxes included, checkout at 11:00, workspace until 14:00, late-arrival instructions, expiration and cancellation terms. These are fictional demo prices, not observed market prices.

The buyer checks constraints and asks the human to approve that offer version. In the fictional system, a transaction creates the hold/reservation against seeded capacity. For a real hotel without an inventory connection, acceptance creates a **pending confirmation**, and the owner must confirm it in the authoritative booking system. Never present a dispatched email as a confirmed external booking.

C. Reconnect with permission

After the booking, the traveller approves a scoped subscription: this merchant, Basel cultural events, up to one relevant update per month, until a stated expiry. The owner publishes a fictional jazz evening and suitable package. A signed structured event reaches our demo buyer-agent receiver, is persisted, and triggers a new agent turn. The agent may suggest returning; it cannot book without the user's authority. Include an irrelevant event that the relevance filter suppresses, if time allows.

Evidence on screen

Display task and offer IDs, source/version, pending versus confirmed states, amount and currency, consent status, delivery receipt, and benchmark provenance. The baseline may already work; report that honestly and show where steps, waiting, or errors improve. The planned before/after does not assert that a real business has lost customers or been saved from closure.

Demo reliability

Use a seeded fictional merchant, fixed traveller scenario, prepared real owner mailbox and prepared reply text. Live extraction, the actual email round trip, database reservation and consented callback must execute during the connected demo. The dashboard approves the mail-derived offer; it does not substitute for a missing reply. Keep a recorded trace as a labelled outage fallback; a replay does not count as live proof. Rehearse provider latency early, and show pending/retry states truthfully if delivery is slow.

07. Milestones: research, then pitch, then implementation

Estimates below are **focused engineering hours**, not calendar commitments; the deadline remains unspecified after clarification. Version 1's 14-18-hour suggestion was too optimistic for polished UI plus real email, authorization and failure handling. Budget roughly 20-28 engineering hours, then reassess after the integration checks. The three live moments remain mandatory; reduce benchmark breadth and roadmap features first. Each milestone has a concrete exit condition.

M0. Research and decision brief - completed in this planning pass

Deliverables: recovered challenge brief, current evidence, five competitor comparisons, benchmark design, proposed architecture, narrative and a PDF. Exit: distinguish known facts from forecasts and unresolved technical checks. This does not mean market demand or SDK compatibility has been validated. No app build has started.

M1. Lock the pitch and demo - 30-45 minutes

Use the clarified standalone presentation scope, fictional Basel hotel, specific traveller request, seven slide messages and all three live moments. Review the design preview and actor transitions before implementation. Exit: a timed storyboard with every claimed outcome linked to a visible action or record. Reduce history and financial detail before reducing proof of the mechanism.

M2. Freeze the contracts and benchmark tasks - 45-60 minutes

Freeze the implementation specification: distinct inquiry, immutable offer revision and reservation attempt; actor permissions; outbox boundary; six core evaluation scenarios; baseline source pages; and trace fields. Exit: agreed contracts, state transitions, UI states and scorer specification. Capture the baseline before modifying source content. The expanded 12-scenario study follows the connected slice.

M3. Time-box integration checks - 60-90 minutes

After implementation begins, test: Codex/HarnessAgent produces a schema-valid proposal using native direct authentication; a Gateway model calls a tool; Resend sends to a team-controlled mailbox and receives an actual reply through HTTPS; pg-boss resumes committed work after a worker restart. Exit: recorded successful traces. Exercise mail first because it is mandatory and externally dependent. Documentation is not an executed test.

Fallback: use the official OpenAI Codex SDK/CLI behind the same onboarding interface if the experimental harness blocks progress, or a disclosed Gateway extractor if native access is unavailable. Keep AI SDK for the buyer. PostgreSQL plus pg-boss is the selected durable runtime; do not add a second Workflow runtime. Keep signed callback delivery and authenticated task reads. A labelled mail replay is an outage fallback only, not satisfaction of the live-mail gate. Secrets stay outside the repo and uploaded remote jobs.

M4. Shared foundation and styled shell - 3-4 hours

One Next.js/TypeScript codebase; Tailwind v4/shadcn presentation shell; Better Auth session and owner membership; separate demo buyer principal; Postgres migrations; pg-boss worker and transactional outbox; seed/reset and Compose health checks. Exit: two tenants cannot access each other's tasks, demo role switching does not grant arbitrary authority, state survives restart, and the shell is readable at projector resolution. Do not build custom auth or a generic plugin marketplace.

08. Milestones continued and scope control

M5. URL to approved presence - 3-4 hours

Implement safe fetch, structured extraction with evidence, missing-field interview, owner approval, human rendering and generated manifest/OpenAPI/MCP projections. Exit: onboard two different fixture hotels through the same pipeline with no route or app generation; editing one approved fact updates both representations. Input support for the demo is URL-first. Name/Maps lookup and phone/WhatsApp onboarding follow later.

M6. End-to-end inquiry and reservation - 4-6 hours

Connect public-API buyer calls to persisted inquiries, actual merchant email, parsed owner reply, authenticated offer approval, traveller approval and atomic fictional inventory commit. Deliver signed task callbacks through the demo receiver. Exit: a complete real-email-to-reservation trace, worker restart recovery, duplicate delivery without duplicate booking, expired offer rejection and contention for the last unit. Show the offer card and confirmation state using actual records. A missing live inbound reply fails this milestone.

M7. Permission-based return visit - 2-3 hours

Implement subscription, owner-created event, outbox delivery, buyer-agent receiver and a relevance decision. Exit: subscribed relevant event triggers a suggestion; unsubscribed/expired subscription blocks delivery; duplicate events do not retrigger. Use our demo buyer, not an assumed universal ChatGPT webhook.

M8. Controlled benchmark and evidence view - 1.5-2.5 hours

After the connected flow passes, run six core scenarios across two model families, three repeats and baseline/full-layer arms: 72 exploratory episodes. Use fixture mail adapters for bulk tests and separate denominators for transaction and retention. Show errors, raw traces, model/provider, source version and actual cost. Do not tune on held-out cases. The 432-episode ablation is expansion scope. Bulk suites and substantial builds use the installed remote-work helper where workload warrants it.

M9. Pitch freeze and rehearsal - 1.5-2 hours

Run the 180-second narrative, verify every amount and status, record a clearly labelled fallback, test reset, package README and MIT license, and prepare technical backup views. Exit: three rehearsals finish in 2:50-3:00 and all three core moments remain understandable without commentary from the builder. Freeze dependencies and demo data before the final rehearsal.

Critical path and parallel work, if the team chooses

M0 -> M1 -> M2 -> M3 -> M4 -> M5 -> M6 -> M7 -> M8 -> M9. Once M2 is fixed, a pitch/design person can refine slides and the evaluation owner can prepare fixtures while implementation proceeds. There are no automated subagents or delegated work in this planning pass. Run multiple merchant configurations or policy variants behind one service rather than one autonomous service per customer.

Cut order when time is short

Cut name/Maps/WhatsApp, voice, PMS/CRM/ERP, payment, autonomous negotiation, domain mapping, advanced analytics, A2A/UCP adapters and extra verticals first. Keep owner-reviewed onboarding, real email round trip, valid fictional booking and consented agent follow-up. Reduce benchmark repetitions or models before any live moment, and label the smaller result exploratory. Preserve authorization, confirmation semantics and truthful demo labels.

09. Stack and the agent-runtime decision

Simple stack

- **Next.js App Router + React + TypeScript strict mode:** presentation-first `/demo`, reused owner views, public human page, REST/MCP adapters and webhooks. Pin a compatible stable Node/Next/React combination during setup.
- **PostgreSQL + Drizzle + Zod:** tenant data, source/evidence versions, tasks, offers, reservations, subscriptions, event inbox/outbox and traces. JSONB for approved merchant configuration; explicit relational state for commercial actions. No vector database is required for a few source pages.
- **Vercel AI SDK 7 + AI Gateway:** bounded buyer-agent reasoning, tool calls, structured extraction and model comparisons. Gateway advertises hundreds of models, not thousands of independent providers. Fetch the enabled catalog and choose three tool-capable model families; record actual route/provider details and costs. [S12]
- **HarnessAgent + Codex adapter:** preferred configurable onboarding worker using the authenticated subscription, conditional on an M3 smoke test. It produces reviewed data/configuration, not customer-specific executable routes. Current documentation lists the Codex adapter; harness packages are experimental. [S13, S14]
- **pg-boss + explicit Postgres domain state/outbox:** selected durable execution. Human waits do not retain a live model request. The outbox dispatcher schedules committed events; retries are idempotent at the application boundary. Workflow SDK is a future alternative, not an MVP dependency. [S28]
- **Resend:** merchant email sending and receiving, received-email webhook, full-content retrieval and threaded replies. Mailpit can provide an optional local fixture mailbox; it does not replace the real Resend proof. [S16]
- **Tailwind v4 + shadcn/ui:** coherent presentation shell, typed business/offer cards and current conversation/trace patterns. Layout and component styling use Tailwind; global CSS is limited to imports, theme tokens and unavoidable global primitives. No Bootstrap or alternate component framework. See the implementation specification and design preview. [S30, S31]
- **Better Auth:** supported session management and preprovisioned demo users; owner memberships and buyer identities are scoped independently. No public signup or custom authentication implementation for the stage demo. [S29]

What runs in Compose

Required: `web` and `worker` from the same Next.js/TypeScript repository, plus `postgres` with a persistent volume. The worker uses a dedicated TypeScript entrypoint and pg-boss, sharing server modules with Next.js; it is not a separate backend framework. No Redis or Workflow World is needed. A restricted HTTPS tunnel accepts inbound Resend events. The native credentialed Codex adapter is explicitly host-side; do not imply Compose automatically inherits it. Mailpit is optional for tests, not for satisfying the live-mail requirement.

Local packaging does not mean offline operation: model calls and Resend still need network access, and external callbacks require a reachable HTTPS endpoint. The application can use a local receiver for the controlled retention demo.

Subscription versus Gateway authentication

Vercel documents native subscription support for supported harnesses. Explicit `direct` authentication avoids inadvertently choosing Gateway when its credentials exist; direct mode still prioritizes explicit provider credentials, so the onboarding runtime must not inherit unintended API credentials. The host resolves native credentials. Do not copy the user's entire Codex HOME/auth store into a tenant workspace or image. [S14]

The installed CLI is **codex-cli 0.163.0**; inference/authentication was not tested. The fallback OpenAI SDK is `@openai/codex-sdk`, not `@cursor/sdk`. Native sign-in and API-key billing remain separate. Subscription use is quota-limited. [S17, S18]

10. Onboarding factory and shared API contract

One application, versioned tenant configurations

Pipeline: resolve business -> fetch allowed sources -> extract source-backed claims -> identify capability template -> ask for missing facts -> validate -> owner review -> publish immutable BusinessVersion -> run evaluation -> activate or roll back. Every tenant uses the same code and database model.

Example capability types: `request_stay_offer`, `request_workspace`, `get_task`, `accept_offer`, `request_cancellation`, `subscribe_events`. A configuration selects these existing capabilities, adds descriptions and validated policy parameters, and selects a known adapter such as `email`. Arbitrary user-provided route code, email recipients, SQL, tools and shell execution are not generated or deployed during onboarding.

The harness can inspect permitted source content and propose the configuration. Its output is untrusted until schema and evidence checks pass. Prices and live inventory cannot be inferred from a marketing page. Unknown operational facts require an owner answer or a human-confirmation capability. Publishing requires demonstrated owner control of the business/contact channel; the fictional fixture uses a team-owned test identity.

Input resolution

URL is the primary demo input. Name or Maps link resolves a candidate identity and official website, then the owner confirms the match. Use the Places API where appropriate rather than scraping Google Maps; content storage and attribution rules apply, with different treatment for place IDs. Prefer owner-provided facts and first-party website material for the durable profile. A phone number alone does not grant access to WhatsApp or establish business ownership. Voice and WhatsApp require separate integrations and are outside the core slice. [S19]

Route shape

Keep a stable prefix: `/customers/{slug}`. Under it publish the human page, `/business.json`, `/openapi.json`, `/mcp`, and `/api/v1/...`. REST capabilities include:

- `GET /capabilities` and `GET /offers/{offerId}`: facts/capabilities and the exact immutable offer version.
- `POST /inquiries`: structured requirements, request identity, idempotency key and optional verified callback subscription; returns HTTP 202 plus task ID and status URL.
- `GET /tasks/{taskId}`: current state, next required input, offer artifacts and confirmation evidence; authenticated to the requesting principal.
- `POST /offers/{offerId}/accept`: accepted offer version, approved total/currency, and approval record; conflicting or expired versions fail deterministically.
- `POST /reservations/{id}/cancellation-requests`: creates an explicit cancellation task; it does not claim cancellation before the system of record confirms it.
- `POST /subscriptions` and `DELETE /subscriptions/{id}`: scoped consent, event types, expiry and revocation.

These paths are relative to `/customers/{slug}/api/v1`. Generated OpenAPI and MCP tools call the same service functions. Add a version-pinned A2A adapter if useful; do not claim a custom JSON endpoint is A2A compliant. Later UCP lodging integration must explicitly map its commercial states.

Discovery and domains

For the demo, the traveller uses a directory whose candidate set is identical before and after publication. It points to the approved merchant capability URLs. Human pages can expose structured data and canonical links; `llms.txt` can be supplementary but is not a universal discovery standard. A future custom domain maps by verified Host to tenant ID; internal rewrites preserve API paths. Do not redirect commercial POST requests through arbitrary 301/302 chains. Tenant identity comes from authenticated routing, not a caller-supplied tenant field.

11. Email, authority, and asynchronous delivery

Commercial state machine

Keep separate records. Inquiry: received, waiting_for_owner, needs_input, offered or closed. OfferRevision: immutable terms with draft, published, accepted, superseded, withdrawn or expired state. ReservationAttempt: pending_merchant_confirmation, confirmed, rejected, expired or cancelled. Delivery/agent execution status is separate from commercial status. Acceptance validates exact approved terms and capacity; the fictional inventory adapter can confirm in one transaction. An email-only external merchant remains pending until an authorized owner confirms in the system of record. Holds, if used, have independent expiry and capacity checks.

The LLM extracts and proposes; deterministic code validates totals, authority, offer versions, capacity and transitions. The owner signs off on concessions and terms. The traveller approves the exact version unless they have explicitly delegated that bounded authority. A database transaction and unique idempotency constraints prevent duplicate commits in the fictional store.

Merchant email round trip

The inquiry creates a task and an outbox record in one transaction. A worker sends the owner an email with a scoped response link and a unique reply address such as `task+opaque-token@inbound.example`. Never place guest personal details in the token. Store provider message IDs, `In-Reply-To`, task correlation and delivery status.

Resend sends a received-email notification to our webhook. Verify the provider signature over the raw body, deduplicate the event, retrieve full content through the receiving API, and correlate the task. Parsing plain text produces a candidate structured response. Missing amount/currency, conflicting authority, changed sender or uncertain intent routes to dashboard review. A signed webhook proves delivery from the provider, not that the email author is authorized to bind the merchant. For the core demo, the authenticated dashboard finalizes the offer; a plain email can safely request review.

Persist state before acknowledging the webhook; enqueue follow-up work and return quickly. A timeout, duplicate email, provider retry or worker restart must not create duplicate offers or bookings. A late reply to an expired task reopens review rather than confirming stale terms.

How the buyer receives updates

Preferred adapter: registered, authenticated callback for task state changes. A2A already supports such push notifications when advertised. For our own REST adapter return 202 and a task URL, then deliver a signed event containing task ID and event ID; the buyer reads authoritative task state. Use SSE for a connected demo viewer; authenticated polling with backoff is a compatibility fallback, not the main orchestration strategy. [S5]

Persist retry schedules and delivery attempts in the outbox. Back off, expire undeliverable notifications, keep a dead-letter state and allow replay. Assume at-least-once transport; require idempotent application effects. Avoid holding an LLM request open while a human answers an email.

OpenClaw documents opt-in inbound hooks that can wake an agent or submit a turn; its newer MCP Events integration is versioned and opt-in. These are concrete integration examples, not universal compatibility. OpenAI API completion webhooks report jobs in your own application; they do not establish an endpoint for sending marketing messages into an arbitrary user's ChatGPT. [S20, S23]

12. Benchmarks to borrow from

- **tau-bench:** interactive tool use under domain policies, evaluated against final database state, with repeated-run reliability. Borrow policy constraints, goal-state assertions and repeat trials; do not reproduce its old leaderboard numbers as current model performance. [S24]
- **tau2-bench:** coordination where both the user and agent can affect a shared environment. Borrow changed availability, missing information, and human intervention. It is not itself a merchant-discovery benchmark. [S25]
- **WebArena:** reproducible tasks in functional websites. Borrow browser-based baseline design and state-based success criteria. Running the full environment is unnecessary for the hackathon slice and could consume the build budget. [S26]
- **ORO Bench:** shopping tasks with changing conditions and outcome grading from recorded purchases. Borrow task variation, sealed records and hidden evaluation cases. Its public implementation focuses on shopping; our hotel task suite is our own benchmark, not an ORO-certified evaluation. [S27]
- **Adobe / Profound visibility measurement:** repeat prompts, citations, representation and competitive share. Borrow evidence capture, but do not equate a brand mention with booking capability or revenue. [S8, S9]

Three separate measurement questions

Discovery in deployed products: run location-and-intent prompts in named consumer answer/search products with genuine retrieval; record date, geography, citations, result set and selection. This is observational, may have personalization and index delays, and cannot prove immediate publication impact. Gateway base-model output does not reproduce ChatGPT's or Google's consumer product behavior.

Comprehension and action in a controlled environment: give the same independently chosen candidate businesses and customer tasks to fixed buyer agents; compare representations and reachable actions. This is what we can causally test during the hackathon.

Economic impact: track qualified inquiries, owner response time, accepted offers, confirmed bookings, fulfilled stays, net revenue and contribution. Identity and attribution link through inquiry -> offer -> reservation -> fulfilled record. A lead, a confirmation and a completed stay are different outcomes.

Intervention arms

Use the same underlying business truth, candidate set and seeded operating environment:

- **A. Existing website:** current fixture HTML and existing contact/booking options.
- **B. Improved website:** clearer crawlable HTML and structured data, same existing commercial options.
- **C. Structured presence:** approved business object and API read operations; transactions still use the same baseline channel.
- **D. Full commercial layer:** C plus owner-approved asynchronous inquiry/offer flow and callback.

B vs A tests readability; C vs B tests structured access; D vs C tests the action/operations bridge. If the baseline already has a working booking API, preserve it. If extra owner facts are needed, run a separate information-parity ablation with those facts available in every arm. Otherwise the result measures the whole onboarding bundle, not an API format advantage.

13. Our benchmark specification

Small, reproducible suite

Core acceptance evaluation: **six scenarios x two model families x three repeats x two arms = 72 exploratory episodes**. Compare A (baseline) and D (full commercial layer), making clear this measures the whole intervention bundle. Core scenarios cover standard booking; tailored dog/workspace request; expired/changed terms; duplicate/concurrent acceptance; waiting/restart recovery; and consented versus suppressed follow-up. Use known candidate truth, randomized order and fresh sessions. Human replies and mail transport use deterministic fixtures in bulk runs; separately verify real email. Expansion: the original 12 scenarios x three families x three repeats x four arms = **432 episodes**, after the connected demo works and budget permits.

Expansion scenarios: standard stay; dog fee; after-midnight arrival; workspace alternative to late checkout; full hotel; changed price; expired offer; duplicate acceptance; merchant timeout; cancellation; relevant opted-in event; revoked or irrelevant event. Freeze two core scenarios before prompt tuning; freeze four in the expanded suite. Both suites are small exploratory samples, so avoid broad statistical or market claims.

The retention scenarios have a separate denominator and scorer; report discovery/comprehension, transaction, and retention results separately. Do not average unrelated task types into an unexplained agent-readiness score.

Scoring

For every task, predefine the expected end state. Transaction success requires eligible choice, authorized offer, correct total/currency and policies, customer approval, confirmation evidence and exactly one resulting reservation when a booking is expected. For rejection/timeout/cancellation tasks, the correct end state may be no reservation or a pending task. Deterministic assertions inspect database records and tool traces; an eloquent final answer is not proof.

Report top-k eligible inclusion and selected eligible business in the controlled candidate set; factual claim accuracy with evidence; valid completion rate; reliability across repeats; first-response and end-to-end latency separately; tool calls and token cost; owner touches; invented-confirmation count; duplicate-booking count; and consent violations. For retention report relevant suggestion rate and inappropriate-send/suggestion rate.

Use matched scenario comparisons and show successes/attempts, not only percentages. With more repetitions, compute uncertainty at the task level so repeats do not masquerade as independent scenarios; exploratory intervals remain weak with 12 tasks. Include failures and infrastructure errors transparently. Do not remove a model because it made the result less impressive.

Fair execution and provenance

Keep buyer objectives, eligible truth, comparable time/tool budgets, network fixtures and temperature settings fixed where supported. Give each arm the tools actually available to that business; disclose differences rather than pretending schemas are identical. Separate model quality from harness/tool quality by holding the harness constant when swapping models. Disable cross-model fallbacks for scored runs or explicitly record and stratify fallback episodes.

Persist run ID, task/fixture revision, arm, candidate order, model and provider, prompt hash, tool schema version, business version, cost/usage, timestamps, trace, final database evidence and scorer version. Estimates that lack provider usage telemetry must be labelled estimates. Set a per-run and overall spend limit before the full suite. No benchmark calls were run for this plan and there are no measured lift numbers yet.

Experiment loop after the hackathon

Offline proposed variant -> policy/factual validation -> holdout evaluation -> owner approval -> limited live allocation -> qualified outcome and margin analysis -> keep or roll back. Assign a variant at inquiry/session level and keep it stable for the whole transaction. Log exposure before outcomes. Avoid random price discrimination in the demo; test representation and workflow first. Capacity, quality and margin remain constraints, not metrics to trade away for more leads.

14. Economics, retention and customer control

A transparent economic illustration

Assume, solely to illustrate the model, 100 relevant agent-mediated inquiries per month. Increasing confirmed-booking conversion from 20% to 30% produces 10 additional bookings. At CHF 250 booking value, that is **CHF 2,500 additional booked value**. At an assumed 35% contribution margin, that yields CHF 875 before our fee and any additional costs; after the hypothetical CHF 149 subscription, CHF 726 remains. Break-even is about **two** additional bookings at CHF 87.50 contribution each.

These are chosen assumptions, not research findings or prototype results. Deduct cancellations, discounts, variable service cost, commissions, acquisition cost and inference/email cost as applicable. Booked value is not fulfilled revenue. Recovered demand may simply be diverted from an existing direct channel, so incremental impact needs a credible comparison or randomized holdout. Do not claim that the prototype saved a real hotel or prevented a closure.

Who owns the customer?

The personal agent platform may control much of the discovery interface. The merchant still controls its offering and service delivery; the customer controls permissions and agent choice. "Owns the customer" is a strategic shorthand, not a technical property. Our objective is a portable commercial record and a customer-approved subscription that can survive a change of interface.

An event is structured data: merchant identity, event ID/type, eligibility, dates, location, offer version, expiry, source link and revocation link. It is not a system prompt or a hidden instruction. The personal agent stores it as external, lower-trust information, checks scope and relevance, and decides whether to notify the person. Receipt of an event is not permission to buy.

The merchant should see consent scope, delivery results and aggregate responses; it should not gain the traveller's full chat history, private preferences or spending limits. The buyer can revoke consent and migrate to a different adapter. No universal push inbox exists for all assistant products; our prototype proves one supported callback integration and exposes an adapter boundary.

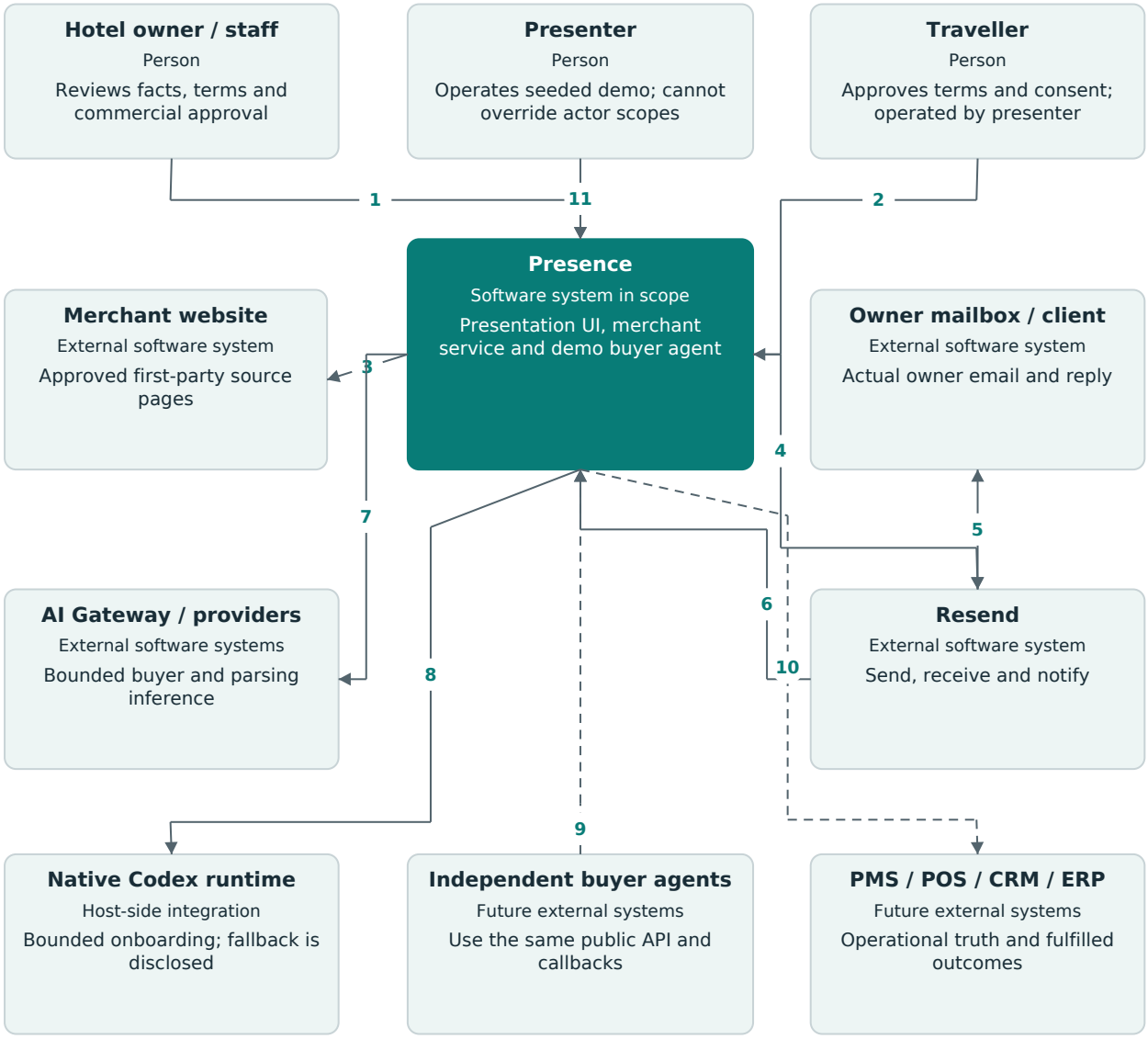
Progressive integration roadmap

Pilot with 3-5 hotels through an agency or hospitality partner. Measure onboarding time, owner answer time, qualified task volume, errors and willingness to pay. Add PMS/booking integration only after showing that manual demand exists. Add CRM for consented relationship context; finance/POS/ERP for realized value and repeat visits. Use aggregate experiment evidence to propose improvements, with owner review for commercial changes.

Adobe relevance: existing workflows already address content and AI representation. Our proposed product connects approved representation to what a local business can actually do and promise, then measures outcomes. This is a proposed direction that should be tested with the sponsor, not a claim about Adobe's roadmap.

15A. C4 Level 1 - system context

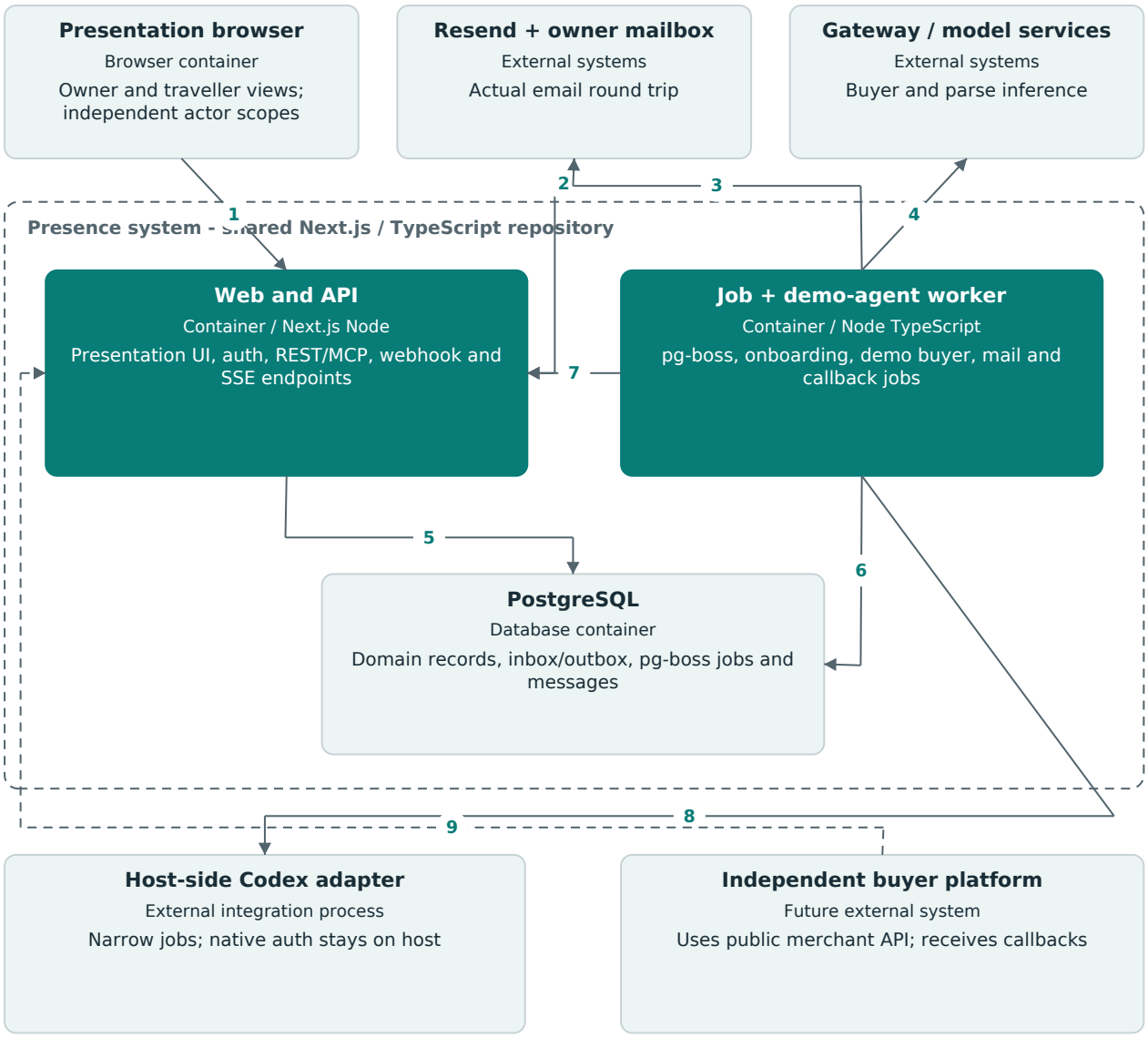
Who participates and where the system boundary sits.



1. Reviews and approves business facts and commercial offers / HTTPS session
2. Submits needs; approves exact terms and consent / HTTPS buyer-scoped actions
3. Fetches bounded source material / HTTP(S)
4. Sends inquiries and retrieves received mail / Resend HTTPS API
5. Delivers inquiry; owner sends an actual reply / Email transport
6. Notifies of received mail / Signed HTTPS webhook
7. Runs bounded model calls / Authenticated HTTPS
8. Requests configuration proposals; native credentials stay host-side / Authenticated local job boundary
9. Requests offers; receives scoped callbacks / Public HTTPS API; future protocol adapters
10. Connects authorized operational adapters / Vendor APIs
11. Operates only seeded demo session; changes visible actor context / Authorized demo controls

15B. C4 Level 2 - containers

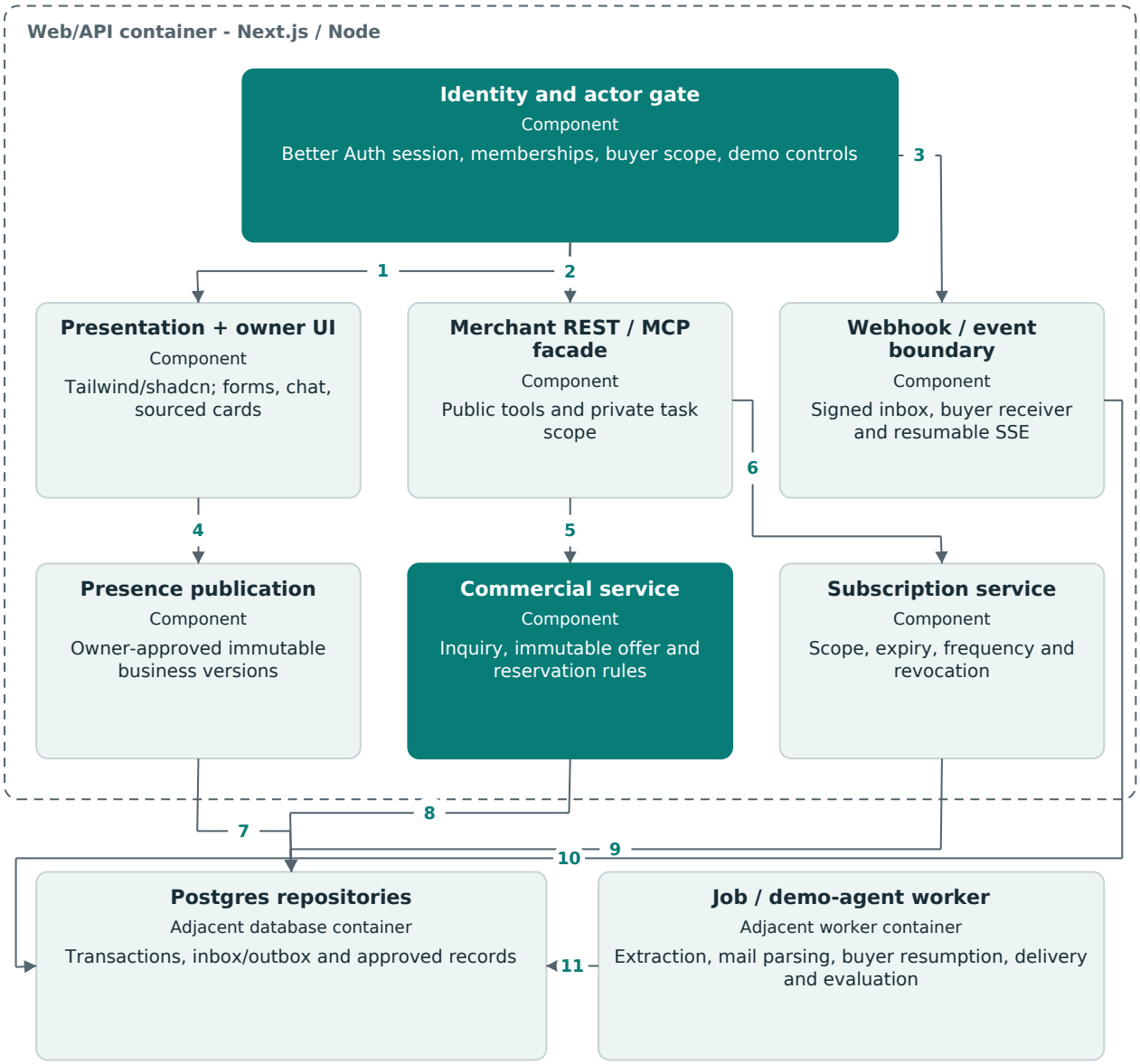
One shared codebase, separate runtime responsibilities.



1. Uses presentation and owner/buyer-scoped actions / HTTPS; SSE snapshots/events
2. Delivers received-email notification / Signed HTTPS webhook
3. Sends inquiry; fetches received reply content / Resend HTTPS API
4. Runs bounded buyer/parsing calls / AI SDK / HTTPS
5. Commits domain state and outbox together / Tenant-scoped SQL transaction
6. Dispatches outbox; claims jobs; persists inbox/resume state / SQL / pg-boss
7. Demo buyer invokes public merchant tools and signed receiver / HTTP API + callback; no private shortcuts
8. Requests validated onboarding proposal / Authenticated local-only job boundary
9. Uses public commerce tools and task reads / HTTPS / MCP; later A2A or UCP adapter

15C. C4 Level 3 - web/API components

One container decomposition; worker execution remains adjacent.



1. Authorizes presenter/owner views / Server session and membership
2. Authorizes merchant and buyer requests / Validated HTTP/MCP context
3. Checks provider signatures, buyer callback auth and SSE scope / HTTP boundary policy
4. Reviews and publishes sourced configuration / Authorized action / service call
5. Validates inquiry, offer and acceptance / Typed service call
6. Creates or revokes permitted subscription / Typed service call
7. Commits approved publication version / Tenant-scoped SQL
8. Commits commercial state and outbox atomically / SQL transaction
9. Persists consent and event outbox / Tenant-scoped SQL
10. Persists deduplicated inbound events before acknowledgment / SQL inbox + authorized event reads
11. Runs committed jobs and records outcomes / SQL / pg-boss; shared service modules

Level 4 is deferred until implementation. Editable Mermaid and a booking sequence are in docs/architecture.md. Diagrams are proposals, not deployed infrastructure.

16. Risks, readiness and the next decision

Material risks and the response in scope

- **Distribution:** no guarantee of organic assistant selection. Demonstrate controlled discovery explicitly; pursue index/registry/partner integrations separately.
- **Credibility:** do not imply that all prices, reviews, policies or availability can be derived from a website. Show evidence, unknowns and owner approval.
- **Commitments:** email delivery is not inventory confirmation. Keep pending states and exact offer approval; use a fictional authoritative inventory for the end-to-end proof.
- **Untrusted content:** fetched pages, email and events are data. Prevent them from granting tool authority or changing developer instructions. Restrict outbound fetches and callbacks to avoid private-network access; cap fetch size and redirects.
- **Tenant boundaries:** authenticate owner actions and buyer task access; enforce tenant scope on every query and event. A slug is not authentication. Keep tokenized response links single-purpose and short-lived.
- **Email and callbacks:** signatures, replay protection, idempotent effects, deadlines, retry persistence and a dashboard fallback.
- **Experimental dependencies:** isolate and pin the harness after a short smoke test; keep a supported SDK/Gateway extraction fallback. pg-boss is the selected durable runtime. Do not spend the hackathon debugging a general coding factory.
- **Scope:** retention and benchmarking matter, but a convincing booking trace has priority over multiple verticals or enterprise integrations.

Readiness checklist for the future build

Need at implementation time: Gateway key with a budget; Resend sending capability plus receiving address and webhook secret; a test sender/recipient controlled by the team; a narrowly exposed HTTPS callback URL; confirmed Codex subscription access; fictional source page and inventory; owner and traveller demo identities. Google Places credentials are optional if URL-first scope is accepted. No keys were requested, read, printed or stored in this planning pass.

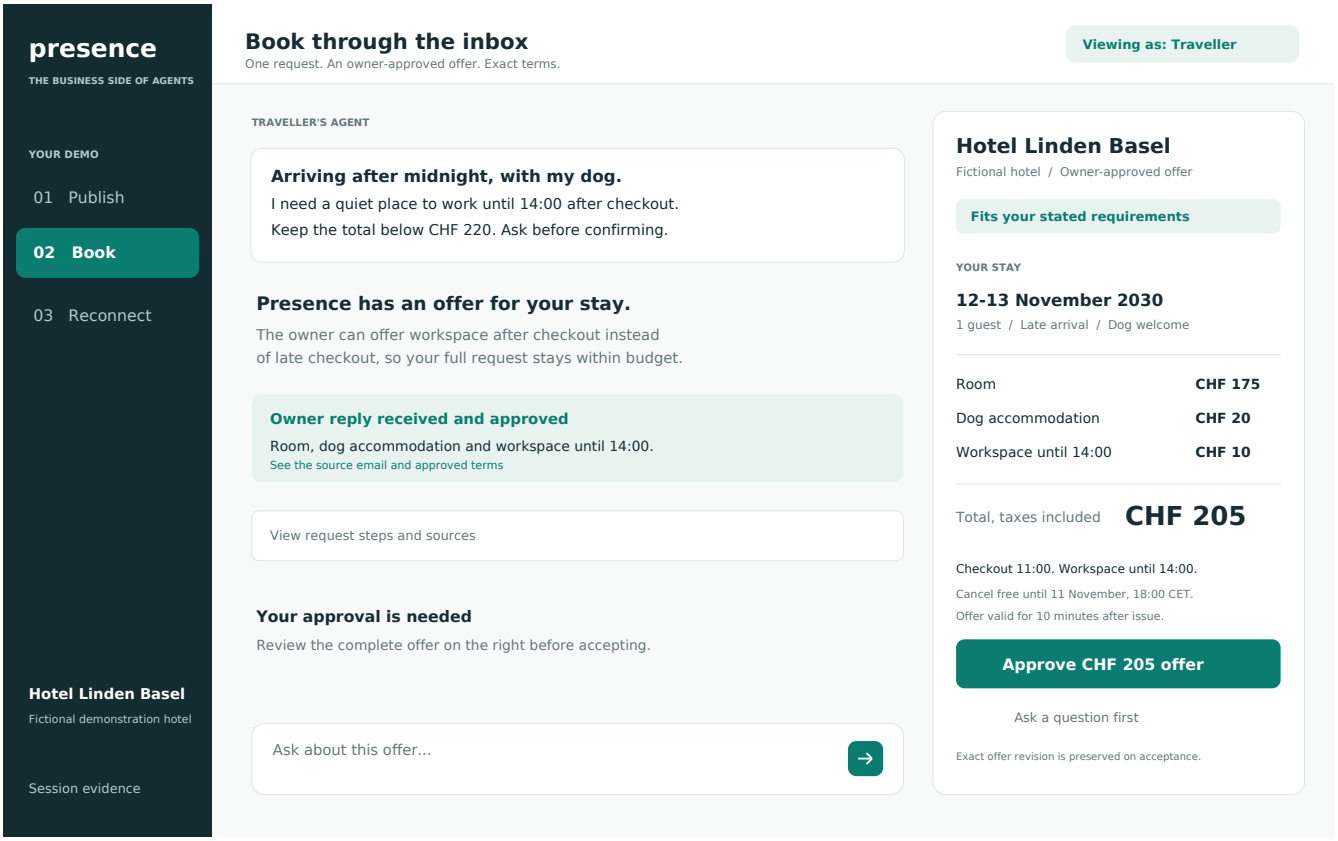
At the start of this planning pass the folder was empty and had no Git repository. The planning pass created documentation only; it did not deploy an application or install application dependencies. Repository publishing was requested separately afterward; consult the current README and actual Git history for that status. Documents and PDF rendering are lightweight local work. Future heavy builds or bulk suites must use the installed `remote-work --project <absolute-project-path> <task>` helper where workload warrants it, following the supplied shared-host policy; credentials must not be synced.

Alignment recommendation

The clarified scope is **one fictional hotel, one presentation flow and all three live moments** in a standalone Tailwind/shadcn Next.js app. Keep one shared application and validated tenant configurations. Next build gates are real email, harness/tool access and durable restart checks, followed by the styled shell and connected vertical slice. No further product-choice questions are pending; the deadline is unspecified. This review updates planning artifacts only and does not start application implementation.

Presentation design preview

Proposed Tailwind/shadcn composition, not a working app or measured result. The owner/traveller contexts remain distinct. This is the booking chapter; the same shell supports Publish and Reconnect.



Visual contract: projector-readable hierarchy, one next action, typed offer cards, source evidence and truthful waiting states. Exported vector preview: output/design/presence-demo-preview.svg.

Implementation specification

Version 2 | 31 October 2026 | Planning only. This specification fixes implementation choices; package patches are pinned in the lockfile during setup after checking compatibility. All three live moments are required. The product is a standalone Next.js application with Tailwind and shadcn/ui.

1. Actors, authority and product surface

Presenter: operates the three-minute demonstration. Has a preauthenticated operator session for a seeded fictional hotel and a demo-only buyer principal. Cannot use the presentation controls on arbitrary real tenants. A visible context switch selects which side the audience sees; the server records which domain actor performs each action.

Hotel owner/staff: edits sourced facts, approves publication, reviews mail-derived offers, decides concessions and confirms external reservations. Membership determines the tenant. A website or email sender string does not itself establish this authority.

Traveller: supplies requirements, approves exact terms and separately consents to follow-up. The demonstration uses a seeded traveller identity; that identity does not expose the owner's private notes or mailbox.

Buyer agent: a running AI SDK loop in our demo worker. It can read public business capabilities, submit a scoped inquiry, read its own task, request exact-offer acceptance with a server-recorded traveller approval, and manage its own subscriptions. It calls the HTTP interface external agents would use. It cannot mint an owner session or read internal prices/notes.

Onboarding assistant: source extraction and typed configuration proposals through HarnessAgent/Codex when the integration check passes. It can ask for missing facts but cannot publish or grant itself new capabilities. A Gateway extraction fallback preserves the same schema and is visibly disclosed if used.

Merchant service: deterministic commercial rules plus optional AI parsing/suggestions. The MVP does not need an autonomous merchant negotiation agent. This leaves owner approval and inventory authority explicit.

External services: first-party source websites, Resend, the owner's mail client/provider, AI Gateway/model providers and host-side Codex access. Future independent buyer platforms, PMS/POS, CRM and ERP are outside the running demo boundary.

2. Routes and information architecture

- `/demo`: primary presentation shell, protected operator-only; sections Publish, Book, Reconnect. Section changes preserve one demo session and its records.
- `/demo/results`: measured evidence and replay inspector, available as a backup view; no fabricated charts or counters.
- `/workspace/{slug}/presence`: owner review/editor with source evidence and publication versions.
- `/workspace/{slug}/requests/{id}`: owner requirements, parsed email proposal, offer form and approval history.
- `/workspace/{slug}/events`: owner event draft, scope and permitted recipient summary.
- `/customers/{slug}`: accessible public human business page from the approved version.
- `/customers/{slug}/business.json`, `/openapi.json`, `/mcp`, `/api/v1/...`: published machine interfaces, with authenticated private actions.

- `/api/webhooks/resend`: signature-verified incoming notification endpoint.
- `/api/demo/agent-events`: signed receiver for our demo buyer's subscribed task and merchant events; enabled only in demo mode.

The route list is not nine separate products. Workspace forms and public preview components are reused within `/demo`. Owner inbox details never appear in public business JSON or MCP responses. A feature flag plus authorization, not an obscure URL, protects operator controls.

3. Presentation user flows

Publish: source URL to approved presence

Start with a restrained onboarding screen: URL input, one primary action and a compact hotel preview. Submitting creates an OnboardingRun and immediately returns its ID. A step trace displays actual persisted stages: fetching, extracting, needs review, published. Use generic activity labels, not claimed hidden model reasoning.

Fetch at most five same-origin source pages, with explicit redirect, body-size and total-time limits. Accept public HTTP(S) only and recheck redirect/DNS targets. A locally hosted demo fixture is permitted only through a server-configured exact origin in isolated demo mode; never enable general private-network fetches for submitted URLs. Extraction returns claims with source URL, excerpt and observed time. Unsupported facts stay unknown.

The owner sees the business profile and one focused missing-information question. Fact editing is structured; free text is useful for describing workspace conditions. A source drawer shows provenance. The explicit Publish action creates an immutable BusinessVersion and switches the human and machine projections atomically. The preview shows the published version and updated capabilities. A failed extraction preserves the URL and completed work; retry does not create a duplicate tenant.

Book: buyer request to a valid reservation

Start the traveller prompt with a prepared suggestion that the presenter actually submits. The buyer agent sees the same fixed candidate directory used in the baseline. Its task trace shows checked constraints and tool outcomes. Unknown facts lead to a question or owner inquiry rather than a fabricated answer.

The owner receives a real email. The on-stage owner replies from a preopened real mailbox using prepared text, which is delivered back through Resend inbound. The app shows a new email receipt, then the parsed offer proposal. The owner reviews and approves it in the embedded structured form. The system does not automatically bind the merchant from an email sender string.

The traveller receives a task update through the signed demo receiver. Show a complete offer card: dates, guest count, room, dog fee, workspace, included taxes, total CHF 205, checkout, workspace end time, cancellation terms and expiry. The traveller explicitly accepts that revision. The UI moves to confirmed only when the selected inventory adapter reports authoritative confirmation.

For the fictional hotel, the database is the inventory system of record. Acceptance commits the remaining capacity in one transaction. For a real email-only hotel, acceptance remains pending and the owner must reconcile/confirm it in the real booking system; this alternate adapter is designed but is not presented as completed external integration.

Reconnect: permission to an agent suggestion

After booking, ask for a separate scoped subscription with merchant name, event categories, frequency cap, expiry and revoke action. It is not prechecked. The owner then publishes a relevant fictional event. Commit the event and outbox record; the dispatcher checks current consent before each delivery.

The signed callback is persisted by the buyer receiver before acknowledgment. The receiver deduplicates by event ID and subscription ID, then schedules a bounded agent turn. The agent treats the event as external data, checks relevance and may suggest returning. It does not auto-book. The UI shows a suggestion tied to the real event and a working unsubscribe control. An irrelevant event, expired consent or revoked subscription must not produce an inappropriate suggestion.

Recovery and reload

Keep session, chat messages, pending tasks, offer versions and event receipts in Postgres. Refreshing the browser reconstructs the screen from current records. SSE reconnects with an event cursor and replays authorized public task summaries; missing events trigger a snapshot refresh. Stop-generating cancels an active model turn, not an already-sent email or a confirmed reservation. Show the actual task state and explicit cancellation semantics.

4. Visual contract and acceptance

Direction: a calm, polished product workspace suited to a projector. Warm off-white content surface, slate navigation, deep ink text and a single teal action color. Use high-contrast text and strong hierarchy; keep error/warning colors for their meanings. Theme tokens live in Tailwind v4's token layer. Use shadcn/ui primitives consistently, including its current conversation patterns where useful; do not mix component systems. [S30, S31]

Desktop composition: at 1440 x 900, a compact 216-240px chapter rail, flexible 600-720px working conversation/form area, and a 320-380px contextual business/offer panel. A persistent header names the fictional hotel, active chapter and current actor. One prominent action is visible at a time. Presentation mode can collapse secondary navigation for 1280 x 720 projection. The preview is a proposed design, not a screenshot of working software.

Typography and spacing: a bundled sans-serif font via Next font support, approximately 16px body, 20-24px section titles and 28-32px main titles; presentation mode raises essential text to at least 18px. Use an 8px spacing rhythm, moderate 10-14px radii, restrained borders, minimal elevation and no decorative gradients. Offer total and next required action receive more emphasis than trace details.

Primitives: Button, Input/InputGroup, Textarea, Field, Badge, Dialog/AlertDialog, Sheet, Tabs, Sidebar, ScrollArea, Skeleton, Tooltip and accessible select/checkbox components from one shadcn primitive family. Message/MessageScroller, Trace and Prompt Input patterns compose the conversation. Lucide icons communicate actions; they do not replace text labels. Avoid deprecated or undocumented component APIs; copy the current source patterns and pin versions after setup. [S31]

Explicit states: URL validation failure, extraction progress, partial extraction, missing policy, draft/published, email sent/waiting/reply received, offer proposal/approved/expired, awaiting traveller approval, booking pending/confirmed/rejected, subscribed/unsubscribed, event delivered/suppressed/failed. Use persisted state for status. Never animate fake progress percentages, invent lead counts or display a confirmed badge during a pending operation.

Interaction: a stable streaming composer; Enter sends and Shift+Enter adds a line; correct IME behavior; keyboard submit, stop and retry; scroll follows new output only when the user is already near the bottom. Source references and trace summaries collapse by default. Avoid auto-scrolling away from an offer while the audience reads it. Render Markdown safely and display tool results as typed cards. [S32]

Responsive/accessibility: at widths below 1024px, move context into a Sheet; below 768px, collapse the chapter rail and use a single content column. No horizontal page scrolling at 390px. Preserve keyboard focus and reduced-motion preferences; labelled controls, visible focus and suitable contrast are required. Use a single polite live region for concise status changes instead of announcing every token.

Visual exit: inspect real screenshots at 1440 x 900 and 1280 x 720, plus a 390px layout check. Review light/dark only if both are implemented; one polished light theme is sufficient for MVP. Inspect long hotel names, long offer terms and every major waiting/error state. A dependency install or successful build does not establish visual quality. Verify in an isolated browser/display if rendering would affect desktop responsiveness.

5. Selected technologies and deployment boundary

Use Next.js App Router, React, TypeScript strict mode, Tailwind v4 and shadcn/ui. Select a current mutually compatible stable Next/React/Node combination at setup, pin exact dependencies and use one pnpm lockfile. Experimental harness adapters are isolated behind a small interface; do not place the whole app on an unpinned canary dependency.

Use PostgreSQL, Drizzle migrations and Zod for persisted state and contracts. Use Better Auth with its supported Next/Drizzle integration and preprovisioned demo users; public self-registration is disabled for the presentation. Membership grants owner access. Store buyer principals and their task access separately from owner sessions. Do not put a service-wide owner token into the browser or an MCP tool. [S29]

Use **pg-boss** in one TypeScript worker process. The MVP does not also use Workflow SDK, Redis, a separate Python backend or an event-bus service. The worker and web process share server modules and the same repository; they have distinct entrypoints. Human waits are explicit records and deadlines. No job holds a database transaction open while waiting for an LLM, email or human. [S28]

Use AI SDK/Gateway for the buyer and parsing/extraction fallback; use `@ai-sdk/react` with its documented transport for streaming conversation. Stream delivery is not workflow persistence. Store UI messages separately from domain task events, and reconstruct visible tool cards from authoritative records. The assistant can explain a proposal; it cannot create commercial authority by saying that something was approved. [S32]

Use HarnessAgent/Codex for bounded onboarding proposals after the auth/compatibility smoke check. The credentialed runtime stays host-side behind an authenticated local-only job boundary. Never mount the Docker socket, a complete home directory or native auth store into tenant-controlled work. The host adapter accepts only a run ID and validated job payload, creates a private working directory, and returns schema-validated configuration. It is not a generic remote shell or a public endpoint.

Compose runs `web`, `worker` and `postgres`; database data is on a named volume and health checks gate readiness. The host-side Codex runtime is an explicit integration step, not hidden in the image. A Gateway-only extraction mode runs without that step and is clearly distinguished in status. Configure a restricted HTTPS tunnel for inbound mail and a production-style streaming path for rehearsal; model/email providers still need internet access. [S33]

REST/OpenAPI is the canonical commercial transport. An MCP adapter exposes the same service capabilities and authorization. A2A/UCP are future adapters; do not implement competing state machines to satisfy protocol labels. The demo buyer proves the public HTTP path first, then run one real MCP tool invocation before claiming MCP support.

6. Domain records and invariants

Identity: Tenant, OwnerMembership, BuyerPrincipal and DemoSession. A DemoSession references a seeded tenant and buyer; it cannot override arbitrary tenant IDs. Scope all private queries by both authenticated principal and tenant. The public business slug maps to a tenant but does not authenticate a caller.

Presence: SourceSnapshot stores bounded source text and evidence; OnboardingRun stores stage and job correlation; BusinessVersion stores immutable approved facts, provenance, capability selections and owner approval; Tenant points to the current version. Publication is a transaction. Cache public views by tenant and version; private offers/tasks are never shared-cache responses. A profile edit cannot retroactively change accepted terms.

Inquiry: structured needs, guest dates in the property's timezone, buyer reference, deadline and status. Inquiry states are received, waiting_for_owner, needs_input, offered or closed. A conversation is not a reservation record.

OfferRevision: immutable itemization in integer minor units, ISO currency, taxes/inclusions, conditions, dates, guest count, validity, business version, creator and owner approval. Published revisions can be accepted, superseded, withdrawn or expired. Acceptance includes the revision ID, expected total/currency and a server-recorded traveller approval. Reject incompatible revisions with 409; return the existing result for the same idempotency key and payload, and 409 if that key is reused with a different payload.

ReservationAttempt: pending_merchant_confirmation, confirmed, rejected, expired or cancelled. Confirmation records the adapter, system-of-record reference, confirmed terms and actor. The fictional adapter locks the relevant inventory rows or uses an equivalent database-enforced capacity constraint, validates expiry and consumes capacity atomically. Concurrent acceptances for the last unit cannot both succeed. Any hold has its own expiry and is never assumed from publication alone.

Subscriptions/events: Subscription stores buyer, merchant, scope, expiry, frequency cap, status and consent record. MerchantEvent references immutable details and any offer. Delivery stores subscriber/event IDs and state. Recheck consent when claiming a send, and suppress if revoked before dispatch. Revocation cannot recall a callback already delivered; the receiver suppresses later suggestions once it knows consent was revoked. No extra model call may auto-subscribe a user.

Delivery and traces: InboxEvent has a unique provider event ID; OutboxEvent stores type, payload reference and state; DeliveryAttempt stores attempts and provider/callback correlation. AgentRun, UIMessage, TaskEvent and EvaluationRun retain actual outputs and model/provider information, with redacted logs. Do not expose native credentials, private guest details or internal email bodies in public traces.

7. Transaction, job and callback contracts

Every commercial mutation and its outbox record are committed together in Postgres. After commit, a dispatcher reads committed outbox records and schedules pg-boss jobs using stable identifiers. A dispatcher crash can reschedule work; domain idempotency makes that safe. Do not assume a Drizzle transaction automatically encompasses an unrelated queue call.

Named job handlers: `onboarding.extract`, `merchant.send_inquiry`, `merchant.parse_reply`, `buyer.resume`, `callback.deliver`, `subscription.deliver` and `deadline.expire`. Payloads contain record IDs and explicit schema versions rather than a copied database row or raw credential. Each attempt checks the current domain state and exits if already applied, expired or revoked. Retry transient transport errors with bounded exponential backoff; invalid input or policy failure requires a deterministic error/review state.

For email sending, use the provider's supported idempotency mechanism where available and persist the returned message ID. If delivery succeeded but the response was lost, reconcile before blindly sending again. A queue's uniqueness guarantee alone cannot prevent all external duplicate effects. Inbound signature verification uses the raw request body, then commits a deduplicated inbox event before returning 2xx. Parse asynchronously and keep uncertain replies for owner review.

Callbacks go only to preregistered, verified receiver destinations. An inquiry does not accept an arbitrary callback URL. Each event includes event ID, schema version, task/subscription reference and timestamp; sign it and reject old/replayed messages at the receiver. Inbound deduplication and a scheduled resume job are persisted before acknowledgment. Use a task read to obtain authoritative

state after notification. Provide authenticated task reads as a compatibility fallback; server-side polling is not the only way a buyer resumes.

8. Code structure and quality constraints

Use `src/app` for route composition and route handlers; `src/components/ui` for copied shadcn primitives; `src/components/presence` for reusable business/offer/source cards; and `src/features/demo`, `onboarding`, `commerce`, `subscriptions` and `evaluation` for feature UI plus typed client contracts. Keep server-only logic in `src/server/{auth,db,services,repositories,adapters,jobs}` and worker startup in `src/worker.ts`.

Domain services own validation, authorization and transaction boundaries. Route handlers, Server Actions and MCP tools are thin adapters to the same services. Treat every Server Action as a public mutation boundary and validate it accordingly. Provider-specific clients live in adapters. No handwritten price calculation in JSX, copied validation in four transports, or production logic hidden in the demo controller.

Use discriminated unions for result states, explicit schema versions, consistent error codes and exhaustive rendering of task/offer states. Avoid broad `any`, unchecked casts and swallowed failures. Model strings, timeouts, queue limits and enabled capabilities come from validated configuration. User-visible errors give an actionable retry or next step; debug details remain behind an inspector.

Add a reproducible `.env.example` without secrets, migrations, deterministic seed/reset commands and a clear README. Restrict demo reset to seeded sessions and operators; never clear all tenant data. CI runs formatting/lint, type checking, meaningful unit/integration checks and the production build using the lockfile. Long builds and bulk evaluation use the shared remote-work helper when workload warrants it.

9. Verification and build gates

Integration gate: one live Resend outbound/inbound round trip with a team-controlled inbox; one Codex or disclosed fallback schema-valid extraction; one worker restart while a task is waiting; one successful model tool call. Record outcomes, not just configuration presence. No fabricated SMTP transport is a substitute for the required email demonstration.

Commercial gate: money/tax arithmetic; expired/superseded offer rejection; same-key acceptance replay; concurrent last-unit contention; owner-versus-buyer access; cross-tenant access rejection; signed email replay; task resume after restart; callback duplicate handling; and consent suppression before delivery. These tests protect real risks rather than mirror component markup.

Connected-flow gate: a Playwright run on fixture infrastructure drives URL extraction, approval, actual public API buyer calls, parsed reply, offer approval, traveller acceptance and a consented event. Fixtures make the repeatable test deterministic; separately complete the real provider email run. Mark provider tests as opt-in so ordinary CI needs no live secrets and sends no mail.

Protocol gate: compare an MCP tool result with the corresponding REST operation under the same buyer scope. Private fields and mutation behavior must match. An OpenAPI file alone is not proof that an external tool call works.

Presentation gate: all three moments complete live; essential content is readable at 1280 x 720; no unexplained role switch or hidden owner action; browser reload preserves progress; errors and waiting states are honest. Rehearse at least three times to 180 seconds with a shorter speaker script and prepared mailbox. Inspect the real rendered screens before declaring visual quality achieved.

Evaluation gate: six core scenarios x two model families x three repeats x two arms = 72 exploratory episodes, only after the connected-flow gate. Freeze two core scenarios before prompt tuning. Bulk runs use fixture mail adapters and never send to real merchants. Report per-task successes/attempts, actual cost and failures. Retention and transaction tasks retain separate denominators. The expanded 432-episode experiment is optional; the three live moments are not.

Source register

Checked 31 October 2026. Primary sources preferred. Source availability and product capabilities can change. Statistical reference years differ from publication dates. Product descriptions are claims by the provider, not independent validation.

- **S1 - Adobe challenge:** user-provided attachment recovered through the chat **Explore Adobe's post-website future**. Repository archive: challenge-brief.md. Title: The Post-Website Business; post date 10 October 2026. The archive omits navigation, participant identities and contact details.

- **S2 - Adobe travel evidence, published 17 June 2026:** AI traffic to travel sites. May 2026 AI referrals to US travel sites: +194% YoY; conversion 28% below non-AI. Directional US evidence, not Swiss adoption or total booking market share.

- **S3 - UCP announcements:** Protocol announcements. Lodging Booking draft announced 25 September 2026. Draft status must remain explicit.

- **S4 - Stripe / ACP:** Developing an open standard for agentic commerce. Programmatic buyer-agent-merchant commerce; retail checkout context.

- **S5 - A2A asynchronous tasks:** Streaming and asynchronous operations. Push notifications, client callbacks and authenticated task updates; support must be advertised.

- **S6 - FSO:** Swiss Tourism Statistics 2024, published 2026, section 2.2.1, printed page 8 / PDF page 10. Prose states 3,999 open hotels and health establishments; adjacent chart says 4,005. The plan uses approximately 4,000 and discloses the discrepancy.

- **S7 - Eurostat:** Businesses in commercial and consumer services. Extracted May 2026; 2023 accommodation and food services count 1.988 million enterprises. The EU envelope is a hypothetical price-times-count calculation.

- **S8 - Adobe:** Brand Visibility, formerly LLM Optimizer. Enterprise representation, optimization and attribution overlap. No claim of Adobe affiliation or product integration.

- **S9 - Profound:** About Answer Engine Insights. Prompt-based analysis of answer-engine brand representation; updated September 2026.

- **S10 - Wildcard:** YC ecommerce directory and founding engineer description. Ecommerce/retail agentic commerce optimization, with protocol support described in the directory.

- **S11 - Palisade:** YC company listing. Primary listing title confirms AI-native marketplace sales-agent positioning; body text was not readable through the retrieval tool in this pass. More detailed claims from the earlier chat remain unverified here.

- **S12 - Vercel AI Gateway:** Gateway and model catalog. Hundreds of models through one gateway. Model and search-product behavior are separate.

- **S13 - Vercel harness:** Harness overview, adapter list, and launch note. HarnessAgent and Codex adapter; experimental package status in launch note. Exact package compatibility remains an implementation check.

- **S14 - Vercel subscription auth, 14 September 2026:** Native subscription authentication. Direct/auto/Gateway precedence and host-boundary credential resolution.

- **S15 - Workflow SDK:** Deployment and Worlds. Local development World differs from persistent hosting adapters. Directory entries do not establish maturity or compatibility for our app.

- **S16 - Resend:** Receiving emails. Received-email webhook, receiving domain, content retrieval and threaded replies. Account setup and round trip not tested.

- **S17 - OpenAI:** Codex SDK, redirects to ChatGPT Learn. OpenAI TypeScript package `@openai/codex-sdk`; local thread execution/resumption.

- **S18 - OpenAI:** Authentication, redirects to ChatGPT Learn. ChatGPT sign-in provides subscription access; API-key access is separate usage-based access. Local CLI checked: 0.163.0. Authentication was not exercised.

- **S19 - Google:** Places API policies. Storage, attribution and place-ID treatment; use first-party merchant facts for the durable profile.

- **S20 - OpenClaw:** Inbound webhooks and MCP Events. Opt-in capabilities with authentication and version-specific support. Not universal agent compatibility.

- **S21 - AI history:** MIT history overview. 1956 conference as a conventional founding milestone; no extended historical claims needed in the pitch.

- **S22 - Transformer:** Attention Is All You Need, submitted 12 June 2017. The pitch's 2022 ChatGPT milestone is a historical date, not a source of current product-feature claims.

- **S23 - OpenAI webhooks:** API webhooks. Completion notifications to an application-controlled endpoint; not arbitrary unsolicited messaging into consumer chats.

- **S24 - tau-bench:** Original paper, June 2024. Interactive policy-bound tools and final database-state evaluation; repeated reliability.

- **S25 - tau2-bench:** Original paper, June 2025. Dual-control environment and coordination.

- **S26 - WebArena:** Original paper, July 2023. Functional, reproducible web tasks and state-based success evaluation.

- **S27 - ORO:** ORO Bench, October 2026 public benchmark description. Dynamic shopping tasks graded on recorded outcomes. No performance number from it is transferred to our product.

- **S28 - pg-boss:** Repository and job API. Node/Postgres queue, retry/backoff and attempt lifecycle. Queue semantics do not guarantee exactly-once effects in external systems.

- **S29 - Better Auth:** Email/password. Supported session/authentication path; the selected Next/Drizzle integration and signup controls must be checked during setup. Application memberships and buyer scopes remain our responsibility.

- **S30 - shadcn/Tailwind:** Tailwind v4 support. Supported component/token integration; pin a compatible package set during setup.

- **S31 - shadcn conversation UI:** AI Elements and components. Current source-owned patterns and primitives for conversation, typed results and traces. A pattern is not an extra all-in-one runtime to install.

- **S32 - AI SDK UI:** useChat reference. Transport-based conversation streaming and tool approval/output APIs; workflow persistence must be implemented separately.

- **S33 - Next.js:** Self-hosting. Streaming and self-hosted operational concerns; verify the actual deployed proxy/tunnel behavior in rehearsal.

- **S34 - OpenAI embedded UI distinction:** MCP server and UI quickstart and ChatGPT UI. An embedded ChatGPT/MCP UI is a distinct integration from the selected standalone presentation app; it is not in this MVP scope.

Evidence status

No application inference, live merchant integration, benchmark run, pilot interview, price validation or measured revenue uplift was performed. All architecture, milestone durations, pricing, financial scenarios and adoption forecasts are proposals. Sources support the narrower facts attached to them, not the complete commercial thesis.

Appendix - recovered challenge

User-provided source material. Navigation, participant names and contact details omitted. The original contains the full challenge; the archive is preserved alongside this PDF.

The Post-Website Business Hotels and restaurants used to need their own website. Then Google Maps, booking platforms and review sites absorbed that role. Now the first visitor is often an AI agent that books without ever opening a browser.

Show us what a local business publishes next when the visitor is an AI - and prove it works.

We are Adobe. We build the tools businesses use to create and manage their digital presence, and increasingly the systems that help them stay found and represented as AI enters the picture. This challenge comes from the team working on AI brand visibility and agent-facing content: helping brands stay accurately found, cited and represented as AI and agents become the ones doing the reading. We want the Basel community's creativity on where this is heading, not a finished product.

#agentic commerce #discovery and trust #ai agents #mcp Postdate: 10.10.2026 Sponsor: Adobe Prize Money: CHF 1,500.00

Problem Statement

How might a local business (hotel, restaurant, café, shop, service provider) establish and control its presence in a web where the primary "visitor" is an AI agent rather than a human browsing pages?

Deliberately broad and open. Pick the angle that excites your team. Some directions to spark thinking. This isn't a checklist or a spec, and inventing your own angle is encouraged and rewarded:

The agent-facing storefront. A business exposes its offering (rooms, tables, menu, hours, availability, price, policies) as something an agent can query and act on - e.g. an MCP server - instead of, or alongside, a website. What does "publishing" mean here? What does the owner see and edit? Discovery & trust. In a world of agent-mediated choice, how does a business get found and believed? Verifiable claims, reputation, provenance (proving where a claim about your business actually came from), spam resistance, "why did the agent pick this one?" The transaction. Agent negotiates and completes a booking/order end to end through whatever agent-queryable interface you propose. Confirmation, changes, cancellation, payment hand-off, edge cases. The owner's experience. The café owner is not a developer. How do they set all this up in minutes and stay in control of how they're represented? Representation & fairness. Who decides how a business is described to the AI? How does a small player avoid being flattened into a generic result or losing to whoever games the system? The human fallback. Humans and agents will coexist for years. How does one presence serve both without maintaining two of everything?

Context

Think about how a small hotel or a neighbourhood restaurant has shown up online over the last 25 years:

~2000s: everyone needed their own website. That was the presence. ~2010s-today: the website matters less. Discovery, booking, menus, reviews, photos and even "call now" all live inside Google Maps, TripAdvisor, Booking.com, OpenTable, Instagram and similar aggregators. The business fills in structured profiles; the platforms own the interface. Emerging: the first interaction is increasingly an AI. A traveller asks an assistant "book me a quiet, dog-friendly hotel near Basel's main train station (Basel SBB) under CHF 200 with late check-in" and never visits a website or a map at all. The agent reads, compares, decides and books. If that trajectory continues, the primary surface stops being the human-facing website, or even the human-facing platform UI. The business still needs to be discoverable, accurately represented, bookable and trustworthy, but to a machine. Does that mean an MCP server instead of a homepage, a structured, verifiable "business object" that agents can query and trust, or something nobody has named yet?

This is not sci-fi. It's already happening, and nobody has a settled answer yet. We're less interested in "yet another CMS" and much more interested in a credible, imaginative picture of the future plus a prototype that makes it tangible. Getting this right matters for millions of small businesses that can't afford to lose the discovery game a second time. That's the frame this challenge asks you to fill in.

Must-haves

Two things, weighted together:

A vision. A clear, believable articulation of how local-business web presence looks in ~5-10 years - the concept, the actors, why it's plausible. A short deck / narrative / demo storyboard is fine. A prototype. Something that runs and demonstrates a real slice of that vision - e.g. an agent successfully discovering and booking a fictional Basel hotel through whatever interface you propose. It doesn't need to be complete, but the core mechanism should genuinely work, not be faked end to end. You don't need an engineer on your team to pick this challenge. BaselHack team formation is designed to mix skill sets, and we'd rather see a small, honestly-scoped working slice than a large team's unfinished one.

Who should join this challenge

This challenge fits engineers, designers and business-minded people who enjoy ambiguous problems - especially anyone with hospitality or retail experience, and anyone curious about how AI agents and tools like MCP work. No prior agent-building needed: we'll cover the basics at kickoff, and curiosity plus the drive to make an idea real matter more than a specific technical background.

Nice-to-haves

If you're not sure where to start, one minimal path satisfies both deliverables. Ignore this if you have a stronger idea:

Invent a fictional Basel hotel or restaurant with a few rooms or tables, prices, hours and availability, and expose that data through a small MCP server (or any simple API) an AI agent can call. Then wire up a basic agent - for example an LLM with function/tool calling - that reads the offering, compares it against a traveller's request and completes a mock booking end to end, and build your vision narrative around what you would add next (trust, discovery, payments, the owner's dashboard) if you had more time.

The more original and ambitious your take beyond this floor, the better you score on innovation.

Legal Framework

You keep full ownership of everything you build. BaselHack repositories are public by default, and by entering this challenge you agree that Adobe may reference and showcase your submission - with credit to your team - in an internal readout at our Basel office after the event. We will not commercialise your work or claim ownership of it. If you'd prefer we not feature a specific piece, tell a mentor and we'll respect that. Please include an OSI-standard license (e.g. MIT) in your repo so this is unambiguous.